

Generate-and-Test Trace Features

Status: independent negative-result proposal; redesign required before promotion.

Abstract

This proposal studies whether a streaming agent with limited feature capacity can replace stale temporal traces after the relevant delay in a prediction problem changes. The generate-and-test idea is central to continual representation learning: an agent should create candidate features, test their usefulness, and discard poor ones. In the current trace-conditioning experiment, utility-based replacement often moves active trace timescales closer to the target delay, but it does not reduce prediction error relative to fixed or random baselines. The result is a useful negative finding: plausible feature parameters are not enough; feature utility must be tied to downstream prediction or control improvement.

Standalone Study Summary

This study tests online feature replacement under a limited feature budget. The RL problem is a streaming delay-prediction task where the useful trace timescale changes, so old features can become stale. The implemented methods compare fixed traces, random replacement, generate-and-test replacement, and an oracle-like diagnostic. The experiment measures absolute prediction error, replacement behavior, feature utility, and post-change recovery. The current evidence is negative: the implemented utility rule replaces features but does not beat simpler random or fixed baselines on prediction error. The next step is to redesign utility so it reflects downstream value or control improvement rather than only local activity.

Research Motivation

A long-lived agent cannot keep every possible feature. Sensor statistics, reward delays, and task-relevant history lengths can change while the agent continues learning. Fixed representations either waste capacity on stale features or require human retuning. Generate-and-test is attractive because it turns representation maintenance into an online algorithmic problem: generate candidate features, estimate their utility from the stream, and replace low-utility features.

The hard question is utility. A feature may look structurally appropriate, such as a trace with a delay near the reward delay, but still fail to improve prediction under the current learning rule, step size, or feature budget. This proposal tests that gap directly.

Research Question

Under a fixed trace-feature budget, can utility-based generate-and-test replacement preserve useful prediction timescales after the reward delay changes?

Hypothesis:

> A useful generate-and-test rule should improve post-change recovery and late prediction error compared with fixed trace banks and random replacement.

The current evidence does not support this hypothesis for the implemented utility rule.

Alberta Plan Connection

The proposal targets Alberta Plan feature finding and continual representation adaptation. It also reflects the Oak-style idea that learned components should be evaluated and replaced based on utility. The experiment is intentionally small and linear so that feature replacement events and timescales are auditable.

Related Work

The Alberta Plan identifies feature finding and generate-and-test as early steps toward continual agents. Recurrent generate-and-test work studies online state-feature discovery. TIDBD and plasticity work motivate adaptive learning parameters but do not solve feature selection by themselves.

Local references:

- - [resources/alberta_plan_related/learning_agent_state_online_2112.15236.pdf](#)
- - [resources/alberta_plan_related/tidbd_1804.03334.pdf](#)
- - [resources/alberta_plan_related/alberta_plan_2208.11173.pdf](#)

Environment

The setting is a trace-conditioning stream:

- - a cue appears;
- - reward follows after a delay;
- - halfway through the stream, the delay changes from 10 to 20;
- - the learner sees the stream once, with no replay or stored dataset.

The feature budget is intentionally small. This creates pressure to choose temporal traces rather than simply include every plausible timescale.

Methods

Compared feature strategies:

- - Fixed tight trace bank.
- - Oracle trace bank.
- - Random replacement.
- - Generate-and-test replacement.

The current generate-and-test rule estimates feature utility online and replaces low-utility features with newly sampled traces. The oracle bank is a diagnostic, not a perfect upper bound; the current run shows that its design also needs validation.

Experimental Design

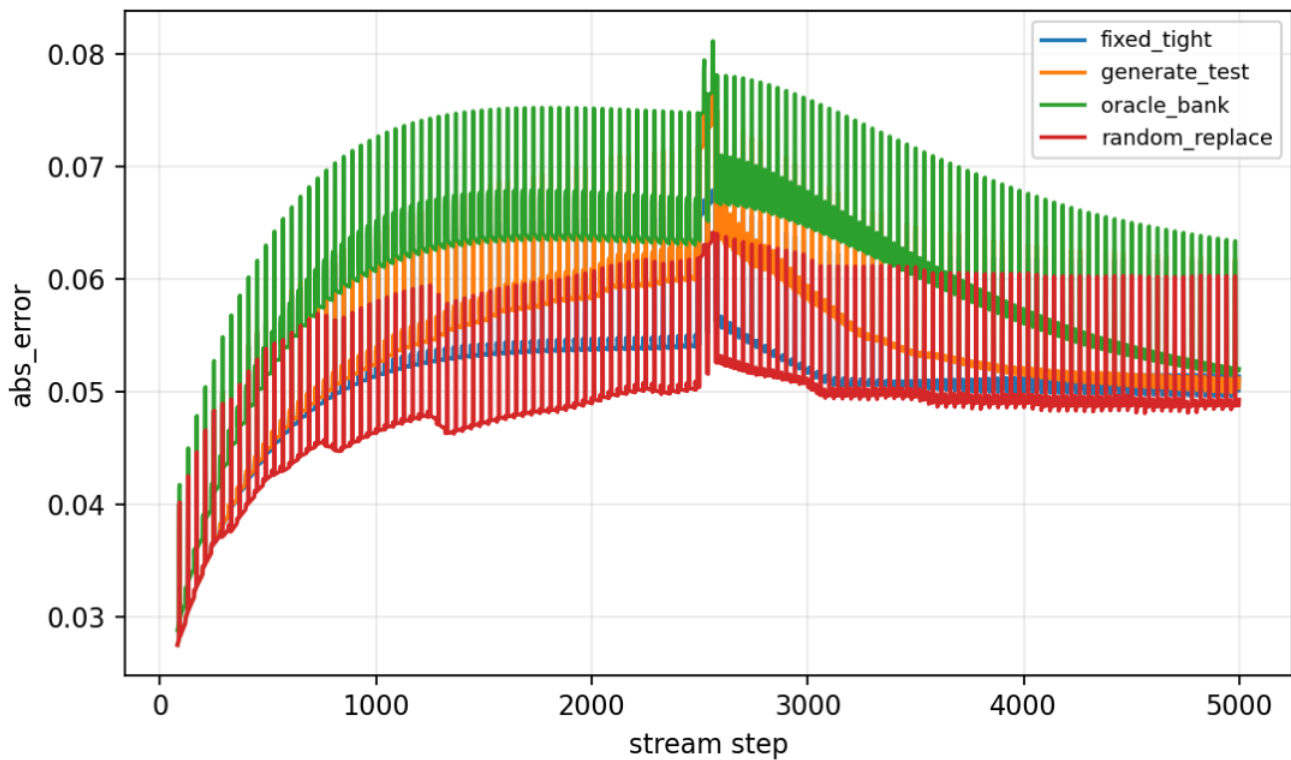
Current main pilot:

- - Feature budget: 4 traces.
- - Delay switch: halfway through the stream.
- - Seeds: 0-4.
- - Steps: 5000.
- - Result path:
[experiments/alberta_core_rl/results/generate_test_features/20260708T154941Z_main.](#)

Metrics:

- - absolute prediction error;
- - closest distance from active trace timescales to target delay;
- - replacement count;
- - recovery-window summaries;
- - pre/post-switch phase labels.

Primary figure:



Delay-shift prediction error by feature strategy.

Results

Generate-and-test often keeps trace timescales closer to the target delay than random or fixed traces. However, this structural improvement does not translate into lower prediction error. Late post-change absolute error is about 0.0518 for generate-test, compared with 0.0512 for fixed-tight, 0.0496 for random replacement, and 0.0545 for the current oracle-bank baseline.

The oracle-bank result is especially important. If an oracle-style trace bank does not clearly improve error, the testbed or feature set is not yet a fair promotion environment for generate-and-test. The study therefore exposes a design flaw rather than a successful representation-learning mechanism.

Analysis

The negative result has two interpretations:

1. The utility measure may be misaligned with prediction error.
2. The environment may be too easy or poorly conditioned, so many trace banks perform similarly.

The result does not refute generate-and-test broadly. It refutes the stronger claim

that the current utility and replacement mechanism is sufficient for this delay-switch stream.

The study also warns against qualitative feature inspection. Seeing a trace timescale move toward the target delay is not enough. The feature must improve a metric that matters: prediction error, recovery time, or downstream control.

Threats To Validity

The current stream has a single delay switch. A serious representation-adaptation paper should include repeated changes and varied delays.

The oracle baseline is not validated as a true upper bound. Before promotion, an oracle feature set must clearly outperform generic baselines.

The feature utility rule is compact and may not match stronger generate-and-test methods.

The task is prediction-only. A downstream control task might value traces differently.

Reviewer Critique And Revisions

Feature-learning reviewer:

- - A feature-generation proposal must show utility, not only plausible feature parameters.

Revision made:

- - Added recovery-window metrics and active-timescale diagnostics.

Strict reviewer concern:

- - The current oracle being worse than random makes the experiment underpowered as a test of feature utility.

Required next revision:

- - Build a validation stream where a known trace bank reliably wins, then test whether generate-and-test recovers it under repeated delay changes.

Conclusion

Generate-and-Test Trace Features is a valid independent negative proposal. It shows that limited-capacity feature adaptation is a meaningful Core RL question, but the current implementation does not improve prediction error. The next version needs a stronger oracle, repeated nonstationarity, and a utility metric tied directly to recovery or downstream control.

Proposal Template Answers

Focused RL question: Under a limited feature budget, can generate-and-test replacement discover trace features that improve a nonstationary delay prediction problem? The setting is a delay-switch trace-prediction stream; the comparison includes random replacement, utility replacement, and oracle-style trace banks. The main metrics are downstream prediction error, active trace timescale, feature survival, and post-switch recovery. Compute is small; fallback is a negative redesign result.

Independent Research Scope

This is an independent negative representation-learning proposal. It does not show that generate-and-test succeeds; it shows that the current utility rule and testbed are insufficient. It should not be merged into Predictive State Plasticity as positive feature-selection evidence.

Evidence Level

Evidence level: negative/redesign. The current utility rule does not clearly beat random replacement, and the oracle trace bank is not strong enough to validate the environment. This is a design failure signal, not a method success.

Experiment Design Rationale

The task is useful only if there is a known feature bank that should win. Without that validation, a generate-and-test failure is ambiguous. The next version must first construct a stream where oracle traces reliably improve downstream error, then test whether utility replacement can recover those traces under a budget.

Reviewer Audit

Reviewer angle	Critique	Action taken	Remaining risk
Representation	Utility replacement does not beat random convincingly. Evidence level is negative/redesign. Needs validated oracle trace bank.		
Experimental design	Oracle is not a clear upper bound. Report requires testbed validation first. Current result cannot evaluate generate-and-test fairly.		
Strict instructor	Do not call plausible feature dynamics a success. Success must be downstream error/control improvement. Needs feature-budget sweep and repeated switches.		

Reproduction

```
cd /mnt/shared-storage-user/yupeng/Core-RL
```

```
PYTHONNOUSERSITE=1 MPLCONFIGDIR=/mnt/shared-storage-user/yupeng/Core-RL/.mplconfig \
/data/yupeng/conda_envs/core-rl/bin/python experiments/alberta_core_rl/scripts/run_experiment.py \
--config experiments/alberta_core_rl/configs/generate_test_features/config_main.json
```